

学号：202300130183	姓名：宋浩宇
实验名称：实验二十-智能问答基础实验——基于大语言模型的问答系统	
1. 实验目的	
1. 理解智能问答（Question Answering, QA）的基本概念。 2. 掌握问答系统的基本组成结构。 3. 学习使用预训练语言模型实现智能问答。 4. 理解上下文信息在问答任务中的作用。 5. 分析大语言模型在问答任务中的优势与局限性。	
2. 实验原理	
智能问答系统的目标是： 根据用户提出的问题，自动生成或抽取正确答案。 例如： 问题：法国的首都是什么？ 答案：巴黎 现代问答系统通常基于预训练语言模型（如 BERT、GPT）实现。 常见问答方式包括： 1. 抽取式问答（Extractive QA） 从给定文本中抽取答案： 文本：北京是中国的首都。 问题：中国的首都是什么？ 答案：北京 2. 生成式问答（Generative QA） 模型直接生成自然语言答案： 问题：什么是人工智能？ 答案：人工智能是一种模拟人类智能行为的技术。 本实验主要使用预训练模型实现基础智能问答任务。	
3. 实验步骤与代码	
实验步骤： 1. 导入 transformers 与 torch 库 2. 加载预训练问答模型 3. 输入上下文文本 4. 输入问题 5. 对文本与问题进行编码 6. 调用模型进行问答推理 7. 输出答案结果 8. 修改问题并重新测试 9. 分析模型输出结果	
代码： <pre># -*- coding: utf-8 -*- from __future__ import annotations</pre>	

```

import os
from pathlib import Path
_WEEK10_DIR = Path(__file__).resolve().parent
_NLP_ROOT = _WEEK10_DIR.parent
_PROXY = os.environ.get("NLP_WEEK10_PROXY",
"http://127.0.0.1:7897")
if os.environ.get("NLP_WEEK10_DISABLE_PROXY", "").strip().lower()
not in ("1", "true", "yes", "on"):
    for _k in ("HTTP_PROXY", "HTTPS_PROXY", "http_proxy",
"http_proxy"):
        os.environ.setdefault(_k, _PROXY)
    os.environ.setdefault("NO_PROXY", "localhost,127.0.0.1,::1")
    os.environ.setdefault("no_proxy", "localhost,127.0.0.1,::1")
if not os.environ.get("HF_HOME"):
    _hf_home = _NLP_ROOT / "_hf_cache"
    _hf_home.mkdir(parents=True, exist_ok=True)
    os.environ["HF_HOME"] = str(_hf_home)
if os.environ.get("NLP_WEEK10_USE_MIRROR", "").strip().lower() in
("1", "true", "yes", "on"):
    os.environ.setdefault("HF_ENDPOINT", "https://hf-mirror.com")
if "HF_HUB_DISABLE_XET" not in os.environ:
    os.environ["HF_HUB_DISABLE_XET"] = "1"
os.environ.setdefault("HF_HUB_DOWNLOAD_TIMEOUT", "600")
os.environ.setdefault("HF_HUB_DISABLE_SYMLINKS_WARNING", "1")
BASE_DIR = _WEEK10_DIR
DATA_PATH = BASE_DIR / "exp-10.1-data.txt"
QA_MODEL = os.environ.get(
    "NLP_WEEK10_QA_MODEL",
    "uer/roberta-base-chinese-extractive-qa",
)
def parse_data(path: Path) -> dict[str, str | list[str]]:
    ctx, qs, ctx_ext, qs_ext = "", [], "", []
    mode = None
    for raw in path.read_text(encoding="utf-8-sig").splitlines():
        line = raw.strip()
        if not line:
            continue
        if line == "[上下文]":
            mode = "ctx"
            continue
        if line == "[问题]":
            mode = "q"
            continue
        if line == "[上下文_扩展]":

```

```

        mode = "ctx_ext"
        continue
    if line == "[问题_扩展]":
        mode = "q_ext"
        continue
    if mode == "ctx":
        ctx = line if not ctx else ctx + " " + line
    elif mode == "q":
        qs.append(line)
    elif mode == "ctx_ext":
        ctx_ext = line if not ctx_ext else ctx_ext + " " + line
    elif mode == "q_ext":
        qs_ext.append(line)
    return {"ctx": ctx, "qs": qs, "ctx_ext": ctx_ext, "qs_ext":
qs_ext}
def _hub_token() -> str | None:
    t = os.environ.get("HF_TOKEN") or
os.environ.get("HUGGING_FACE_HUB_TOKEN")
    return t.strip() if t and t.strip() else None
def download_model(model_id: str) -> str:
    from huggingface_hub import snapshot_download
    token = _hub_token()
    plans: list[tuple[str, bool]] = []
    cur = os.environ.get("HF_ENDPOINT", "https://huggingface.co")
    plans.append((cur, True))
    if cur.rstrip("/") != "https://hf-mirror.com":
        plans.append(("https://hf-mirror.com", False))
    last_err: Exception | None = None
    proxy_keys = ("HTTP_PROXY", "HTTPS_PROXY", "http_proxy",
"https_proxy")
    for endpoint, use_proxy in plans:
        saved = {k: os.environ.get(k) for k in proxy_keys}
        os.environ["HF_ENDPOINT"] = endpoint
        if not use_proxy:
            for k in proxy_keys:
                os.environ.pop(k, None)
        try:
            kw: dict = {"repo_id": model_id}
            if token:
                kw["token"] = token
            print(f"尝试下载 Endpoint={endpoint} 代理={'开' if
use_proxy else '关'}")
            return snapshot_download(**kw)
        except Exception as exc:

```

```

        last_err = exc
        print(f"下载失败: {type(exc).__name__}: {exc}")
    finally:
        for k, v in saved.items():
            if v is None:
                os.environ.pop(k, None)
            else:
                os.environ[k] = v
    if last_err is not None:
        raise last_err
    raise RuntimeError("模型下载失败")
def build_qa_model():
    import torch
    from transformers import AutoModelForQuestionAnswering,
AutoTokenizer
    token = _hub_token()
    kw = {"trust_remote_code": True}
    if token:
        kw["token"] = token
    local_path = download_model(QA_MODEL)
    print(f"模型目录: {local_path}")
    device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
    tok = AutoTokenizer.from_pretrained(local_path, **kw)
    model =
AutoModelForQuestionAnswering.from_pretrained(local_path, **kw)
    model.to(device)
    model.eval()
    return tok, model, device
def predict_qa(tok, model, device, question: str, context: str) ->
dict:
    import torch
    enc = tok(
        question,
        context,
        max_length=512,
        truncation="only_second",
        return_tensors="pt",
        return_offsets_mapping=True,
    )
    offset = enc.pop("offset_mapping")[0].tolist()
    input_ids = enc["input_ids"][0]
    seq_type = enc.get("token_type_ids")
    if seq_type is not None:

```

```

        seq_type = seq_type[0].tolist()
    valid = []
    for i, (cs, ce) in enumerate(offset):
        if cs == 0 and ce == 0:
            valid.append(False)
        elif seq_type is not None and seq_type[i] == 0:
            valid.append(False)
        else:
            valid.append(True)
    enc = {k: v.to(device) for k, v in enc.items()}
    with torch.inference_mode():
        out = model(**enc)
    start_logits = out.start_logits[0]
    end_logits = out.end_logits[0]
    best_score = float("-inf")
    start_idx = end_idx = 0
    max_len = 32
    for s in range(len(valid)):
        if not valid[s]:
            continue
        e_upper = min(s + max_len, len(valid))
        for e in range(s, e_upper):
            if not valid[e]:
                continue
            score = float(start_logits[s] + end_logits[e])
            if score > best_score:
                best_score = score
                start_idx, end_idx = s, e
    answer = tok.decode(input_ids[start_idx : end_idx + 1],
skip_special_tokens=True).strip()
    answer = answer.replace(" ", "")
    char_start = offset[start_idx][0] if start_idx < len(offset)
else -1
    char_end = offset[end_idx][1] if end_idx < len(offset) else -1
    return {"answer": answer, "score": best_score, "start":
char_start, "end": char_end}
def run_qa(tok, model, device, title: str, context: str, questions:
list[str]) -> None:
    if not context or not questions:
        return
    print(f"\n--- {title} ---")
    print(f"上下文: {context}")
    for i, q in enumerate(questions, 1):
        r = predict_qa(tok, model, device, q, context)

```

```

        ans = r["answer"] or "(空)"
        print(f"[{i}] 问题: {q}")
        print(f"    答案:
{ans} score={r['score']:.6f} span={r['start']}-{r['end']}")
def main() -> None:
    print(f"数据文件: {DATA_PATH}")
    print(f"模型: {QA_MODEL}")
    print(f"代理: {os.environ.get('HTTPS_PROXY', '(未设置)')}")
    print(f"HF_HOME: {os.environ.get('HF_HOME', '(未设置)')}")
    print(f"HF_ENDPOINT: {os.environ.get('HF_ENDPOINT',
'https://huggingface.co')}")
    if not DATA_PATH.is_file():
        print("数据文件不存在。")
        raise SystemExit(1)
    data = parse_data(DATA_PATH)
    try:
        tok, model, device = build_qa_model()
    except Exception as e:
        print(f"模型加载失败: {type(e).__name__}: {e}")
        print("提示: 请确认本地代理 127.0.0.1:7897 已开启;模型下载到 HF_HOME 目录, 请保证该盘空间充足。")
        print("    若 C 盘存在不完整缓存, 可删除 models--uer--roberta-base-chinese-extractive-qa 后重试。")
        print("    直连失败可执行: set NLP_WEEK10_USE_MIRROR=1 后改用 hf-mirror.com。")
        raise SystemExit(1) from e
    run_qa(tok, model, device, "任务书上下文与问题",
str(data["ctx"]), list(data["qs"]))
    if data["ctx_ext"] and data["qs_ext"]:
        run_qa(tok, model, device, "扩展上下文对比",
str(data["ctx_ext"]), list(data["qs_ext"]))
if __name__ == "__main__":
    main()

```

4. 实验结果（图表与输出）

输出结果为:

F:\Homework\自然语言处理作业\venv\lib\site-packages\torch\cuda_init_.py:61: FutureWarning: The pynvml package is deprecated. Please install nvidia-ml-py instead. If you did not install pynvml directly, please report this to the maintainers of the package that installed pynvml for you.

```
import pynvml # type: ignore[import]
```

数据文件: F:\Homework\自然语言处理作业\第十周实验\exp-10.1-data.txt

模型: uer/roberta-base-chinese-extractive-qa

代理: (未设置)

HF_HOME: F:\Downloads\huggingface
HF_ENDPOINT: https://huggingface.co
尝试下载 Endpoint=https://huggingface.co 代理=开
模 型 目 录 :
F:\Downloads\huggingface\hub\models--uer--roberta-base-chinese-extractive-qa\snapshots\9b02143727b9c4655d18b43a69fc39d5eb3ddd53
Loading weights: 100%|████████████████████| 199/199 [00:00<00:00, 43934.44it/s]

--- 任务书上下文与问题 ---
上下文: 人工智能是一种模拟人类智能行为的技术。机器学习是人工智能的重要分支。深度学习是机器学习中的一种方法。
[1] 问题: 什么是人工智能?
答案: 人工智能是一种模拟人类智能行为的技术 score=10.220715 span=0-18
[2] 问题: 机器学习属于什么领域?
答案: 深度学习 score=4.346786 span=34-38
[3] 问题: 深度学习是什么?
答案: 人工智能 score=10.993279 span=0-4
[4] 问题: 中国的首都是什么?
答案: 人 score=-8.139488 span=0-1

--- 扩展上下文对比 ---
上下文: 人工智能是一种模拟人类智能行为的技术。机器学习是人工智能的重要分支。深度学习是机器学习中的一种方法。北京是中国的首都。巴黎是法国的首都。
[1] 问题: 中国的首都是什么?
答案: 北京 score=8.812889 span=50-52

这是实验 21 的, 你可以先把这个实验的写了

5. 结果分析

1. 模型回答是否符合语义直觉?

答: 部分符合, 部分明显偏离。第 1 问"什么是人工智能?"抽出的答案为"人工智能是一种模拟人类智能行为的技术", 与上下文首句一致, 语义直觉上正确。第 2 问期望"机器学习属于人工智能领域", 输出为"深度学习", 不符合常识。第 3 问期望说明深度学习是机器学习中的一种方法, 输出仅为"人工智能", 过于笼统且不准确。短上下文中第 4 问"中国的首都是什么?"输出"人", score 为负, 显然错误。扩展上下文后同一问题输出"北京", 与"北京是中国的首都"一致, 符合直觉。整体看, 模型在字面重叠高、答案可直接从文中截取的问题上较稳, 在需要跨句归纳或上下文中无明确答案时容易出错。

2. 哪些问题回答较准确?

答: 当次输出中较准确的是第 1 块第 1 问, 问题"什么是人工智能?", 答案"人工智能是一种模拟人类智能行为的技术", score=10.220715, span=0-18, 与上下文开头整句匹配。扩展上下文块中"中国的首都是什么?"也较准确, 答案"北京", score=8.812889, span=50-52, 对

应扩展段中"北京是中国的首都"。这两问可作为报告中"回答较准确"的示例。

3. 哪些问题容易出现错误？

答：第 1 块第 2 问"机器学习属于什么领域？"输出"深度学习"，`score=4.346786`，将领域关系答错。第 3 问"深度学习是什么？"输出"人工智能"，`score=10.993279` 虽较高，但答案过短且未体现"机器学习中的一种方法"这一关键信息。第 4 问在短上下文中输出"人"，`score=-8.139488`，属于上下文中不存在正确答案时的强行截取。归纳而言，关系型问法、需要完整定义句的问题，以及上下文中无支持片段的问题，更容易出错。

4. 上下文长度是否会影响问答效果？

答：会影响，本次输出有直接对照。短上下文中第 4 问无法得到"北京"，答案为"人"且 `score` 为负。扩展上下文加入"北京是中国的首都。巴黎是法国的首都。"后，同一问题答案变为"北京"，`score=8.812889`。说明是否包含与问题相关的文本片段，直接决定抽取式模型能否给出合理答案。上下文过短或缺少关键信息时，即使模型仍会给出一个 `span`，也可能是无意义片段。

5. 为什么问答系统需要理解上下文语义？

答：抽取式问答并非凭空生成，而是要在给定段落中定位与问题匹配的答案 `span`。若系统不能把握问题所问的对象与上下文各句之间的语义关系，就容易像第 2、3 问那样抽到共现词却答非所问，或在无答案时仍截取任意片段。第 4 问在短上下文与扩展上下文下的对比也说明，系统必须知道"中国的首都"这一问题应到北京相关句中寻找，而不是在仅谈 AI 的句子里乱选。因此理解上下文语义是正确对齐问题与证据句的前提。

6. 生成式问答与抽取式问答有哪些区别？

答：本实验使用 `uer/roberta-base-chinese-extractive-qa`，属于抽取式问答：答案必须是上下文中的连续片段，输出含 `span` 与 `score`，例如"北京"对应 `span=50-52`。生成式问答则由模型自由生成自然语言句子，不受原文逐字限制，可改写或补充，但也更易产生上下文中不存在的内容。本次第 1 问答案几乎整句来自原文，是典型抽取行为；若改为生成式，同样问题可能会输出更短或更口语化的定义句，但不再保证与原文逐字一致。

7. 大语言模型在智能问答中的主要优势是什么？

答：结合本次运行，预训练模型能在与训练分布接近的中文短段落上，较快定位高相关文本片段，例如第 1 问整句抽取与扩展后正确抽到"北京"，无需手工编写规则。相对传统关键词匹配，它对问句与上下文的语义匹配更灵活，`score` 也可用于比较候选 `span` 的置信度。局限在于仍依赖上下文是否包含答案，且对"属于什么领域"这类需关系理解的问题，本次第 2、3 问仍出现高 `score` 但语义错误的情况，说明优势主要体现在证据明确、字面可对的抽取场景。

6. 实验总结

本次加载 `uer/roberta-base-chinese-extractive-qa`，对任务书上下文四问及扩展上下文一问完成抽取式问答推理，程序正常结束。第 1 问与扩展后中国的首都一问结果可用；第 2、3 问及短上下文第 4 问存在明显错例，可用于报告讨论上下文与问法对抽取式 QA 的影响。
使用 LLM 辅助完成代码编写。